

o_que_e_sql

O QUE É LINGUAGEM SQL

1. INDEX

1. [O que é Linguagem SQL](#)
2. [Comandos SQL mais importantes](#)
 1. [RECORDS](#)
 2. [DATABASE](#)
 3. [TABLE](#)
 4. [INDEX](#)
 5. [TRANSAÇÕES](#)
 6. [VIEWS](#)
3. [REFERÊNCIAS](#)

2. CONTEÚDO

1. O que é Linguagem SQL

1. Structured Query Language, ou Linguagem de Consulta Estruturada ou SQL, é a linguagem de pesquisa declarativa padrão para banco de dados relacional (base de dados relacional). Muitas das características originais do SQL foram inspiradas na álgebra relacional. O SQL foi desenvolvido originalmente no início dos anos 70 nos laboratórios da IBM em San Jose, dentro do projeto System R, que tinha por objetivo demonstrar a viabilidade da implementação do modelo relacional proposto por E. F. Codd. O nome original da linguagem era SEQUEL, acrônimo para "Structured English Query Language" (Linguagem de Consulta Estruturada, em Inglês), vindo daí o facto de, até hoje, a sigla, em inglês, ser comumente pronunciada "síquel" ao invés de "és-kiú-él", letra a letra. No entanto, em português, a pronúncia mais corrente é letra a letra: "ésse-quê-éle". [Veja mais...](#)

1. Bons tutoriais que encontrei na web.:

1. [Tutorial w3schools.com/sql](https://www.w3schools.com/sql/);
2. mysqltutorial.org
 1. [basic-mysql-tutorial](#)
 2. [introduction-to-sql-stored-procedures](#)
 3. [mysql-triggers](#)
 4. [mysql-views-tutorial](#)
 5. [mysql-index](#)

2. Dicas:

1. Não há diferença entre maiúsculas e minúsculas como na linguagem c++.
2. A linguagem é interpretada.

3. Existe um padrão na linguagem sql chamado *ansi sql* ou *sql 92*, porém o mesmo não é seguido pelos fabricantes de banco de dados.
4. O sql além de armazenar dados é possível fazer programas a serem executados ao incluir ou excluir ou alterar um registro das tabelas.
5. O mysql não tem opção de depurar programas.
6. [Dicas de como depurar sql](#).
7. [Comparativo entre mysql x postgresQL](#).
8. [MySQL UUID Smackdown: UUID vs. INT para chave primária](#)
9. [Dicas de MySQL](#)

3. 

2. Comandos SQL mais importantes

1. DATABASE

1. [CREATE DATABASE](#) - cria um novo banco de dados

1. Sintaxe:

```
CREATE DATABASE databasename;
```

2. Exemplo:

1. A seguinte instrução SQL cria um banco de dados chamado "testDB":

```
CREATE DATABASE testDB;
```

3. NOTA:

1. *Certifique-se de ter privilégios de administrador antes de criar qualquer banco de dados. Uma vez que um banco de dados é criado, você pode verificá-lo na lista de bancos de dados com o seguinte comando SQL:*

```
SHOW DATABASES;
```

4. [Veja mais...](#)

2. DROP DATABASE - A instrução **DROP DATABASE** é usada para eliminar um banco de dados SQL existente.

1. Sintaxe:

```
DROP DATABASE databasename;
```

2. Exemplo:

1. A seguinte instrução SQL elimina o banco de dados existente "testDB":

```
DROP DATABASE testDB;
```

3. NOTA:

1. *Certifique-se de ter privilégios de administrador antes de descartar qualquer banco de dados. Uma vez que um banco de dados é descartado, você pode verificá-lo na lista de bancos de dados com o seguinte comando SQL: SHOW DATABASES;*

```
SHOW DATABASES;
```

2. A instrução acima não funciona no prompt do banco de dados sqlite.

4. [Veja mais...](#)

3. ALTER DATABASE - modifica um banco de dados

1. Exemplo para alterar o CHARSET e COLLATION de um banco existente:

1.

```
ALTER DATABASE `sua_base`  
    CHARSET = Latin1  
    COLLATE = latin1_swedish_ci; //Não distingue  
    letras minúsculas e maiúsculas e nem caracteres  
    acentuados e com cedilha, ou seja, o registro que  
    contém a palavra "Intuição" será retornado quando  
    houver uma procura pela palavra "intúicao".
```

2.

```
ALTER DATABASE `sua_base`  
    CHARSET = UTF8  
    COLLATE = utf8_general_ci; // Não há  
    distinção entre letras maiúsculas e minúsculas.  
    Buscando por "teste", registros como "Teste" ou  
    "TESTE" serão retornados.
```

4. [DROP DATABASE](#) - Apaga o Banco de dados.

5. A instrução **DROP DATABASE** é usada para eliminar um banco de dados SQL existente.

1. Sintaxe:

```
1. DROP DATABASE databasename;
```

2. A seguinte instrução SQL elimina o banco de dados existente "testDB":

```
1. DROP DATABASE testDB;
```

2. RECORDS

1. **SELECT** - A instrução **SELECT** é usada para selecionar dados de um banco de dados. Os dados retornados são armazenados em uma tabela de resultados, chamada de conjunto de resultados.

1. Sintaxe:

```
SELECT column1, column2, ... FROM table_name;
```

2. Exemplo:

1. Para selecionar todos os campos de uma tabela:

```
SELECT * FROM table_name;
```

3. [Veja mais...](#)

2. **UPDATE** - A instrução **UPDATE** é usada para modificar os registros existentes em uma tabela.

1. Sintaxe:

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

2. Exemplo:

1. A instrução SQL a seguir atualiza o primeiro cliente (CustomerID = 1) com uma nova pessoa de contato e uma nova cidade.

```
UPDATE Customers
SET ContactName='Alfred Schmidt',
City='Frankfurt'
WHERE CustomerID=1;
```

3. **NOTA**

1. *Tenha cuidado ao atualizar registros em uma tabela! Observe a cláusula **WHERE** na declaração **UPDATE**. A cláusula **WHERE** especifica quais registros devem ser atualizados. Se você omitir a cláusula **WHERE**, todos os registros da tabela serão atualizados!*

4. [Veja mais..](#)

3. DELETE - exclui dados de um banco de dados

1. Sintaxe:

```
1. DELETE FROM table_name
WHERE condition;
```

2. A seguinte instrução SQL exclui o cliente "Alfreds Futterkiste" da tabela "Clientes":

```
1. DELETE FROM Customers
WHERE CustomerName='Alfreds Futterkiste';
```

4. INSERT INTO - insere novos dados em um banco de dados

1. Sintaxe:

```
1. INSERT INTO table_name
(column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

2. Se você estiver adicionando valores para todas as colunas da tabela, não será necessário especificar os nomes das colunas na consulta SQL. No entanto, certifique-se de que a ordem dos valores esteja na mesma ordem das colunas da tabela. A sintaxe INSERT INTO seria a seguinte:

```
1. INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

3. A seguinte instrução SQL insere um novo registro na tabela "Clientes":

```
1. INSERT INTO Customers
    (CustomerName, ContactName, Address, City,
    PostalCode, Country)
    VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen
    21', 'Stavanger', '4006', 'Norway');
```

4. A seguinte instrução SQL insere 3 registros na tabela "books":

```
1. INSERT INTO books
    (title, author_id, isbn, year_pub)
    VALUES('The Trial', '1', '0805210407', '1995'),
    ('The Metamorphosis', '1', '0553213695',
    '1995'),
    ('America', '1', '0805210644', '1995');
```

5.  TOP

6.  TOP

3. TABLE

1. CREATE TABLE - cria uma nova tabela.

1. Sintaxe:

```
1. CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....);
```

2. O exemplo a seguir cria uma tabela chamada "Persons" que contém cinco colunas: PersonID, LastName, FirstName, Address e City:

```
1. CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255));
```

2. ALTER TABLE - modifica uma tabela

1. Sintaxe para adicionar uma coluna a uma tabela:

```
1. ALTER TABLE table_name
    ADD column_name datatype;
```

2. O seguinte SQL adiciona uma coluna "Email" à tabela "Customers":

```
1. ALTER TABLE Customers
    ADD Email varchar(255);
```

3. DROP TABLE - exclui uma tabela

1. A instrução DROP TABLE é usada para eliminar uma tabela existente em um banco de dados.

1. Sintaxe:

```
1. DROP TABLE table_name;
```

2. A seguinte instrução SQL elimina a tabela existente "Shippers":

```
1. DROP TABLE Shippers;
```

4. 

4. INDEX

1. Observação: a sintaxe para criar índices varia entre os diferentes bancos de dados. Portanto: Verifique a sintaxe para criar índices em seu banco de dados.

1. [Dicas boas sobre index](#)

2. [Sintaxe do MySql:](#)

1. CREATE INDEX - Cria um índice em uma tabela. Valores duplicados são permitidos:

1. Sintaxe:

```
1. CREATE INDEX index_name
    ON table_name (column1, column2,
    ...);
```

2. A instrução SQL abaixo cria um índice denominado "idx_lastname" na coluna "LastName" da tabela "Pessoas":

```
1. CREATE INDEX idx_lastname
    ON Persons (LastName);
```

3. Se quiser criar um índice em uma combinação de colunas, você pode listar os nomes das colunas entre parênteses, separados por vírgulas

```
1. CREATE INDEX idx_pname
   ON Persons (LastName, FirstName);
```

2. CREATE UNIQUE INDEX - Cria um índice exclusivo em uma tabela. Valores duplicados não são permitidos:

1. Sintaxe:

```
1. CREATE UNIQUE INDEX index_name
   ON table_name (column1, column2,
   ...);
```

3. DROP INDEX - A instrução DROP INDEX é usada para excluir um índice em uma tabela.

1. Sintaxe:

```
1. ALTER TABLE table_name
   DROP INDEX index_name;
```

2.  TOP

5. TRANSAÇÕES

1. START TRANSACTION

1..

2. COMMIT

1..

3. ROLLBACK

1..

4. SET TRANSACTION

1..

5. LOCK TABLES

1..

6. UNLOCK TABLES

1..

6.  TOP

7. VIEWS

1. Uma chave estrangeira é uma restrição que pode ser usada para impor a integridade dos dados. É composto por uma coluna (ou conjunto de colunas)

em uma tabela chamada tabela filho, que faz referência a uma coluna (ou conjunto de colunas) em uma tabela chamada tabela pai. Se chaves estrangeiras são usadas, MariaDB executa algumas verificações para garantir que algumas regras de integridade sejam sempre aplicadas. Para uma explicação mais completa, consulte Bancos de dados relacionais: Chaves estrangeiras .

1. As chaves estrangeiras (FOREIGN KEY) só podem ser usadas com mecanismos de armazenamento que as suportem. O InnoDB padrão e o PBXT obsoleto oferecem suporte a chaves estrangeiras.
 2. As tabelas particionadas não podem conter chaves estrangeiras e não podem ser referenciadas por uma chave estrangeira.
2. As chaves estrangeiras são criadas com CREATE TABLE ou ALTER TABLE . A definição deve seguir esta sintaxe:

```
1.  [CONSTRAINT [symbol]] FOREIGN KEY
    [index_name] (index_col_name, ...)
    REFERENCES tbl_name (index_col_name, ...)
    [ON DELETE reference_option]
    [ON
```

```
reference_option:

    RESTRICT | CASCADE | SET NULL | NO ACTION | SET
    DEFAULT</pre>.
```

3. Exemplos.

1. Vamos ver um exemplo. Vamos criar uma tabela author e uma tabela book. Ambas as tabelas têm uma chave primária chamada author_id também possui uma chave estrangeira composta por um campo chamado author_id, que se refere à author chave primária. O nome da restrição de chave estrangeira é opcional, mas vamos especificá-lo porque queremos que ele apareça em mensagens de erro: fk_book_author.

```
1.  CREATE TABLE author (
        id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT
    PRIMARY KEY,
        name VARCHAR(100) NOT NULL
    ) ENGINE = InnoDB;
```

```
CREATE TABLE book (
    id MEDIUMINT UNSIGNED NOT NULL
    AUTO_INCREMENT PRIMARY KEY,
```

```
title VARCHAR(200) NOT NULL,  
author_id SMALLINT UNSIGNED NOT NULL,  
  
CONSTRAINT `fk_book_author`  
FOREIGN KEY (author_id) REFERENCES author  
(id)  
ON DELETE CASCADE  
ON UPDATE RESTRICT  
) ENGINE = InnoDB;
```

4. Dicas em www.w3schools.com

5.  TOP

3. REFERÊNCIAS

1. Tutorial w3schools.com/sql
2. wikipedia.org/wiki/SQL
3. [Artigos MariaDB para iniciantes](#)
 1. [MariaDB Basics](#)
 2. [Obtendo dados de MariaDB](#)
4. [Restrição SQL FOREIGN KEY](#)
5. [Para saber onde estão as bases de dados fisicamente](#)

